

and require only the minimal runtime (binaries and libraries) needed by the application. This makes them:

- Lightweight (usually measured in megabytes)
- Fast to start (milliseconds)
- More efficient in resource usage, enabling higher density on a single host

Example: While a server might host 5-10 VMs, it could run 50-100 containers depending on the workload.

Complex application architectures: Modern applications are increasingly built using microservices architecture, where functionalities are split into loosely coupled services.

For instance:

- Frontend UI
- REST APIs
- Authentication service
- Message queues (e.g., RabbitMQ)
- Databases (e.g., PostgreSQL)

Managing and deploying such architectures using traditional tools is complex and error prone.

Containers simplify this by allowing each service to run in its own isolated environment. Teams can:

- Develop, test, and deploy services independently
- Reuse base images across multiple services
- Rollback individual services without affecting the whole system

Example: An e-commerce site could have separate containers for user authentication, product catalogue, cart service, and payment gateway.

Cloud-native development and scalability: Cloud providers favour containers because they are portable, stateless, and easily scalable. With container orchestrators like Kubernetes, teams can:

- Automatically scale services based on demand
- Perform rolling updates with zero downtime
- Handle service discovery and load balancing
- Recover from failures automatically

Containers enable cloud-native practices such as Infrastructure as Code (IaC), autoscaling, blue-green deployments, and canary releases.

Example: An image-processing service could automatically scale to 10 containers during high load and shrink to 2 during off-hours, optimising costs.

Improved CI/CD workflows: In modern DevOps practices, continuous integration and continuous deployment (CI/CD) are crucial for fast and reliable software delivery.

Containers enhance CI/CD pipelines by providing:

- Immutable builds – once a container image is built, it won't change.
- Reproducibility – testing in containers ensures the same results in production.
- Simplified testing – each stage of the pipeline (build, test, deploy) uses the same container image.

This leads to fewer surprises in production and faster iterations.

Example: A Jenkins pipeline could build a Docker image from source, run unit tests, security scans, and then deploy it to a staging cluster—all using the same image.

What is containerisation?

Containerisation is the technique of packaging an application along with all its dependencies, libraries, configuration files, and runtime environment into a single executable unit called a container.

This ensures that the application behaves the same regardless of where it is run—on a developer's laptop, a test server, or in production. The key concepts are:

- **Container:** A runnable instance of an image, including application code, libraries, dependencies, and configuration files.
- **Image:** A static specification (blueprint) of what the container should include and how it should run.
- **Docker:** The most widely used platform to create, manage, and run containers.
- **Orchestration tools:** Platforms like Kubernetes manage deployment, scaling, and networking of containers at scale.

Key characteristics of containers

Feature	Description
Lightweight	Containers share the OS kernel, unlike VMs, making them efficient.
Portable	A container can run anywhere a container engine (like Docker) is installed.
Isolated	Each container operates in its own isolated environment.
Immutable	Once created, containers do not change, ensuring consistency.
Fast boot-up	Containers start almost instantly.

Components of containerisation

Component	Role
Container engine	Software like Docker or containerd that runs and manages containers.
Container image	A template that contains everything required to run an application.
Dockerfile	A script that defines the steps to create a container image.
Container registry	Repositories (like Docker Hub, Amazon ECR, GitHub Packages) that store and distribute container images.
Orchestration tool	Tools like Kubernetes or Docker Swarm that manage deployment, scaling, and networking of containers.